

NESL-IT Test Task

Approach

I focused on building a clean and maintainable solution that satisfies all requirements without overengineering. Since the assignment was limited to 2 hours, my goal was to keep the architecture simple while following good engineering practices.

Architecture Decisions

I separated the project into three parts:

- Client (React + TypeScript)
- Server (NestJS)
- Shared Types

This keeps frontend and backend responsibilities separate while ensuring type consistency through shared contracts.

I intentionally avoided microservices and databases because the application has a single domain and does not require permanent data storage. A modular monolith is easier to develop, maintain, and explain for the scope of this task.

Frontend

I used React with TypeScript for maintainability and type safety.

React Konva was chosen because the core requirement is rendering and dragging shapes on a canvas. It provides efficient canvas rendering and built-in drag support.

Zustand manages canvas state because it is lightweight and avoids unnecessary Redux boilerplate. I also used Zustand persistence to support the bonus requirement of restoring state after refresh.

Backend

I used NestJS because it provides a clean modular structure and built-in Socket.io integration.

The backend is split into:

- AI module (prompt → JSON generation)
- WebSocket module (real-time communication)

This keeps responsibilities clear and easy to extend.

Real-Time Synchronization

Socket.io is used to synchronize canvas updates between multiple tabs.

When a shape is moved:

1. The UI updates immediately.
2. The position is sent to the server.
3. The server broadcasts the update to other connected clients.

This provides a smooth user experience while keeping all clients synchronized.

Validation

Since AI-generated responses can be unreliable, I added validation before sending data to the frontend.

The validation ensures:

- Only circles and rectangles are allowed
- Maximum 12 shapes
- Labels are limited to 2 characters
- Shapes remain inside canvas boundaries

Additional Decisions

I added a fallback generator in case the AI service is unavailable. This ensures the application remains functional and demonstrates resilience.

For persistence, I used localStorage instead of a database because the requirements only needed canvas restoration after refresh, making a database unnecessary for this scope.

Summary

My focus was on delivering a reliable real-time application with clean architecture, type safety, validation, and maintainable code while avoiding unnecessary complexity.